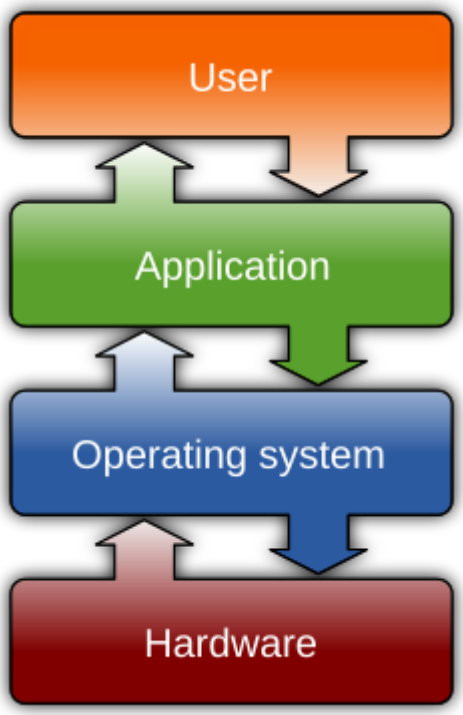
Unit - I
Operating System services and components

1.1 Operating System: concept, functions
1.2 Different types of Operating System: Batch Operating System, Multi-programmed, Time Shared Operating System, Multiprocessor System, Distributed System, Real Time System, Mobile OS (Android OS)
1.3 Command line based Operating System: DOS,UNIX GUI based Operating System: WINDOWS,LINUX, MaC OS
1.4 Different Services of Operating System, System Calls: Concept, types of system calls
1.5 Operating System Components: Process Management, Main Memory Management, File Management, IO Management, Secondary Storage Management

1.1 Operating System

An **operating system (OS)** is system software that manages computer hardware and software resources, and provides common services for computer programs.
An **Operating System (OS)** is a software that acts as an interface between computer hardware components and the user.



Functions of Operating System

1. **User Interface**

   Provides a way for users to interact with the system, either through a **command-line interface (CLI)** or a **graphical user interface (GUI).** It provides a way for users to interact with the system, either through a command-line interface (CLI) or a graphical user interface (GUI).

2. **Process management:**

   Process management helps OS to create and delete processes. It also provides mechanisms for synchronization and communication among processes.

3. **Memory management:** Memory management module performs the task of allocation and de-allocation of memory space to programs in need of this resources.

4. **File management:** It manages all the file-related activities such as organization storage, retrieval, naming, sharing, and protection of files.
5. **Device Management:** Device management keeps tracks of all devices. This module also responsible for this task is known as the I/O controller. It also performs the task of allocation and de-allocation of the devices.
6. **I/O System Management:** One of the main objects of any OS is to hide the peculiarities of that hardware devices from the user.
7. **Secondary-Storage Management:** Systems have several levels of storage which includes primary storage, secondary storage, and cache storage. Instructions and data must be stored in primary storage or cache so that a running program can reference it.
8. **Security:** Security module protects the data and information of a computer system against malware threat and authorized access.
9. **Error Detection and Handling**
   Detects and reports system errors or malfunctions. Takes corrective actions to maintain smooth operation.

1.2 Different types of Operating System:

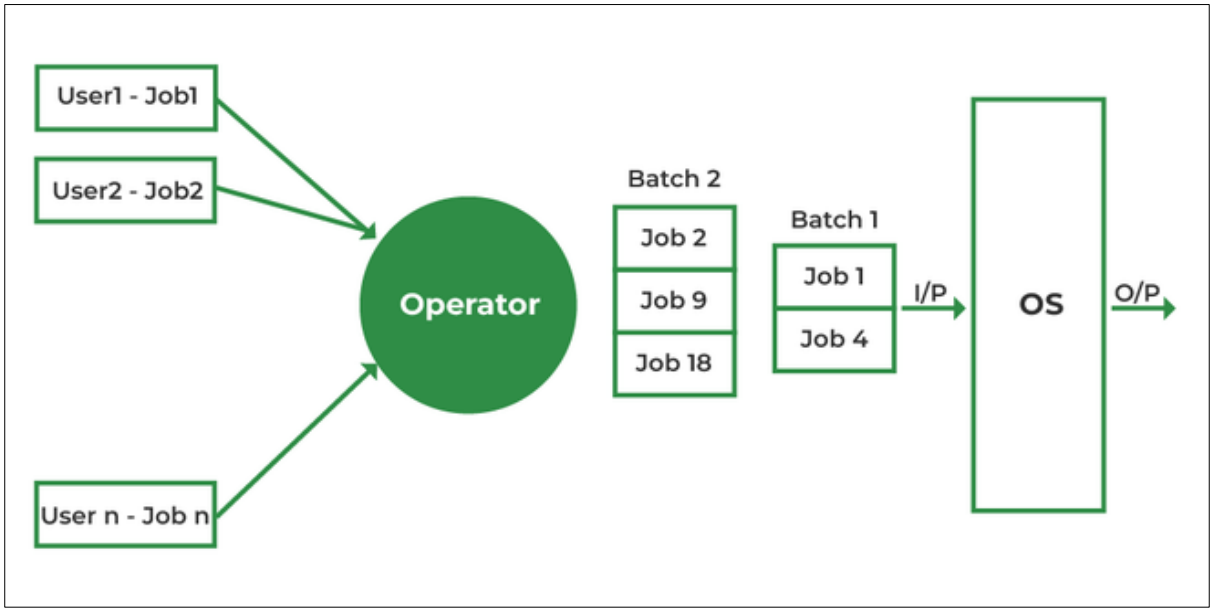
Batch Operating System

- A simple batch system is one of the earliest types of operating systems designed to execute jobs in batches without requiring real-time user interaction.
- A Batch Operating System is one of the earliest types of operating systems, primarily used in the 1950s–1970s**.** It processes batches of jobs without user interaction during execution.
- In batch operating system the jobs were performed in batches. This means Jobs having similar requirements are grouped and executed as a group to speed up processing.
- Users using batch operating systems do not interact with the computer directly. Each user prepares their job using an offline device for example a punch card and submits it to the computer operator.
- Once the programmers have left their programs with the operator, they sort the programs with similar needs into batches.

How It Works

1. **Job Collection**: Users submit jobs to a central location, often via punch cards or tapes.
2. **Batch Formation**: The operator groups jobs with similar requirements into batches.
3. **Execution**: The system processes each batch sequentially, loading jobs into memory and executing them.

4. **Output Delivery**: Results are stored and delivered to users after the entire batch is processed.



Advantages

1. **High Throughput**: Processes large volumes of jobs efficiently.
2. **Reduced Setup Time**: Grouping similar jobs minimizes the overhead of switching between tasks.
3. **Simplified Management**: Automates job scheduling and execution, reducing manual intervention.
4. **Cost Efficiency**: Optimizes resource usage, making it suitable for large-scale data processing.

Limitations

1. **No Real-Time Interaction**: Users must wait until the entire batch is processed to see results.
2. **Long Wait Times**: Jobs with lower priority may experience delays.
3. **No Prioritization**: All jobs are treated equally, which can lead to inefficiencies for critical tasks.
4. **Idle CPU Time**: The CPU may remain idle during I/O operations in simple batch systems.

Use Cases

1. **Payroll Systems**: Calculating salaries for employees in bulk.
2. **Banking Transactions**: Processing large volumes of transactions, such as monthly statements.
3. **Billing Systems**: Generating bills for utilities or telecom services.

Example of Batch Operating System

Some examples of batch-processing operating systems include:

- IBM's z/OS
- Unisys MCP
- and Burroughs MCP/BCS

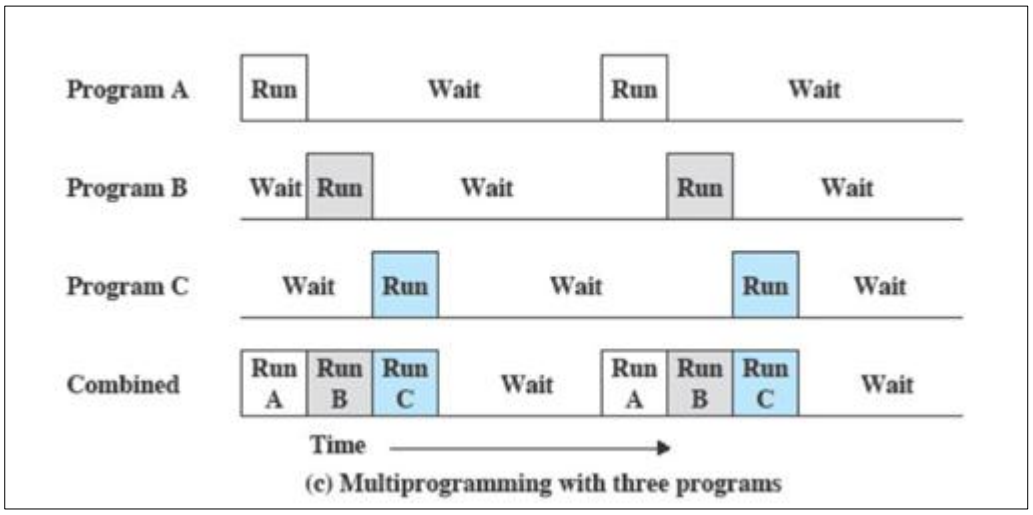
Multiprogramming Operating System

- Multiprogramming OS is an ability of an operating system that executes more than one program using a single processor machine.
- In multiprogramming system, multiple programs are to be stored in memory. The operating system handles all these process and their states.
- Before the Multiprogramming operating system concept, only one program was to be loaded at a time and run. These systems were not efficient as the CPU was not used efficiently. For example, in a single-tasking system, the CPU is not used if the current program waits for some input/output to finish. The idea of multiprogramming is to assign CPUs to other processes while the current process might not be finished.
- The operating system handles all the process and their states. Before the process undergoes execution, the operating system selects a ready process by checking which one process should undergo execution.
- When the chosen process undergoes CPU execution, it might be possible that in between process need any input/output operation at that time process goes out of main memory for I/O operation and temporarily stored in secondary storage and CPU switches to next ready process.
- And when the process which undergoes for I/O operation comes again after completing the work, then CPU switches to this process.
- This switching is happening so fast and repeatedly that creates an illusion of simultaneous execution.

How It Works

- Programs are stored in memory as **processes**.
- When one process needs I/O (like reading a file), the OS switches to another ready process.
- This **context switching** creates the illusion of simultaneous execution.

- Let's P1 and P2 are two programs present in the main memory. The OS picks one program and starts executing it.
- During execution if the P1 program requires I/O operation, then the OS will simply switch over to P2 program. If the p2 program requires I/O then again it switches to P3 and so on.



Advantages

- High **CPU and memory utilization**
- Improved **system throughput**
- Supports **multiple users and tasks**
- Faster **response time** for short jobs

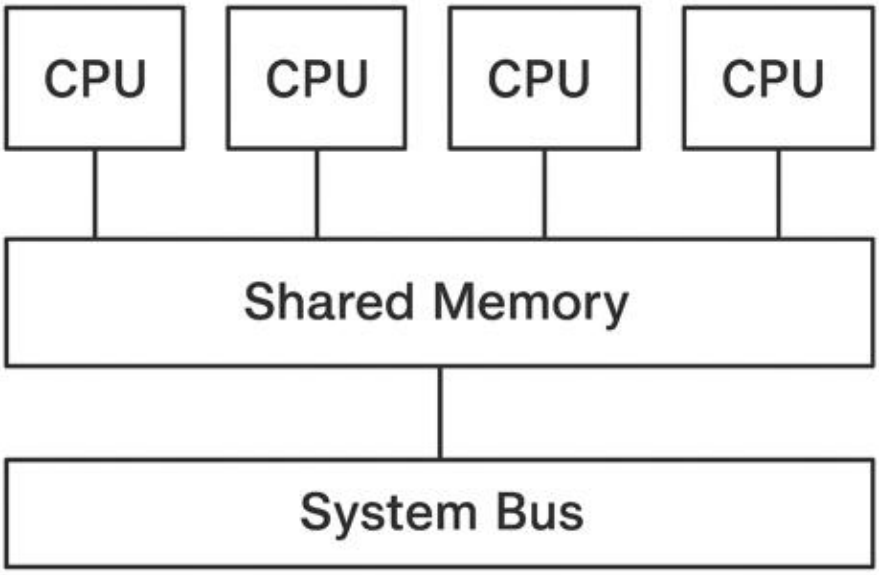
Examples

- Windows
- Linux
- UNIX
- macOS

Multiprocessor Operating system

- A multiprocessor system is a computer system with **two or more processors** (CPUs) that share a common physical memory and are connected via a communication bus or interconnection network.
- These processors work together to execute programs, enabling better performance, reliability, and scalability.
- The main feature, objective, and purpose of the Multiprocessors are to provide high speed at a low cost in comparison to a uniprocessor.
- A multiprocessing operating system is defined as a type of operating system that makes use of more than one CPU to improve performance. Multiple processors work **parallels** in multi-processing operating systems to perform the given task.
- All the available processors are connected to peripheral devices, computer buses, physical memory, and clocks.
- The main aim of the multi-processing operating system is to **increase the speed of execution of the system**. The use of a multiprocessing operating system improves the overall performance of the system.

- For example, UNIX, LINUX, and Solaris are the most widely used multi-processing operating system.



Multiprocessor System

Types of Multi-Processing Operating Systems

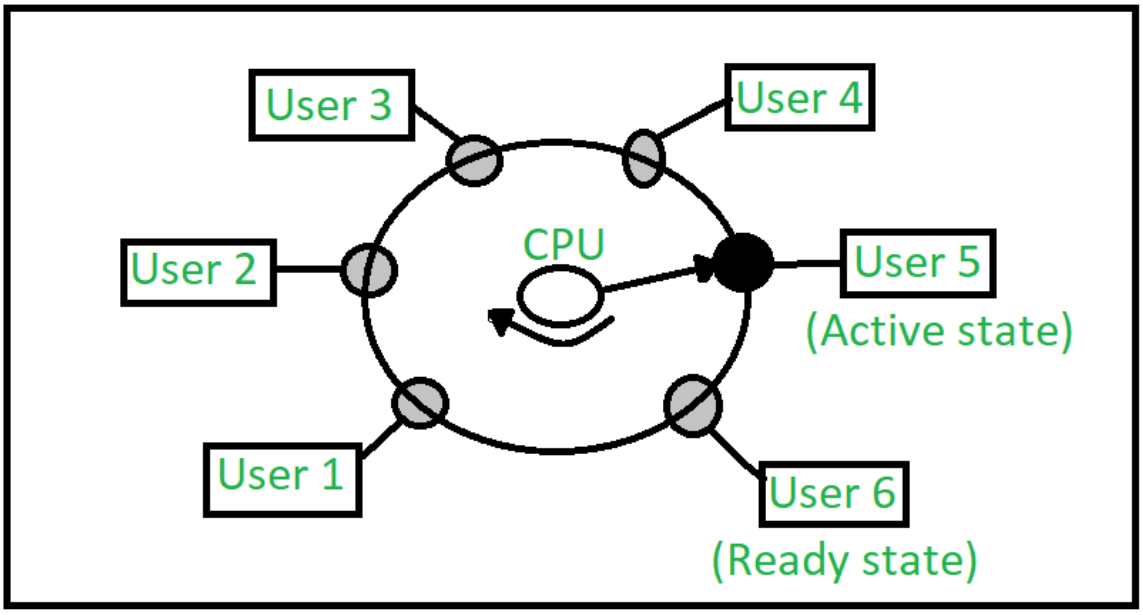
1. **Symmetric Multiprocessing (SMP):**
   - All processors are treated equally.
   - Each process makes its own decisions and works according to all other process to make sure that system works efficiently. With the help of CPU scheduling algorithms, the task is assigned to the CPU that has least burden.
   - Symmetrical multiprocessing operating system is also known as "Shared Everything System" because all the processors share memory and input-output bus.
2. **Asymmetric Multiprocessing (AMP):**
   - In Asymmetrical multiprocessing operating system one processor acts as a master whereas remaining all processors act a slaves.
   - Slave processors are assigned with ready to execute processes by the master processor.
   - A ready queue is being maintained by master processor to provides with processes for slaves.

Advantages:

- **Increased Throughput**: Multiple processors handle more tasks in parallel.
- **Improved Reliability**: If one processor fails, others continue working.
- **Scalability**: Additional CPUs can be added to enhance performance.
- **Faster Execution**: Parallel processing reduces task execution time.

Time Shared Operating System

- **A time-shared operating system** uses **CPU scheduling and multi-programming** to provide **each user with a small portion** of a shared computer at once.
- Each user has at least one separate program in memory. A program is loaded into memory and executes, it performs a short period of time either before completion or to complete I/O.
- This short period of time during which the user gets the CPU is known as **time slice, time slot, or quantum**. It is typically of the order of **10 to 100 milliseconds**.
- Time-shared operating systems are more complex than multiprogrammed operating systems.
- To achieve a good response time, jobs may have to swap in and out of disk from the main memory which now serves as a backing store for the main memory.



In the above figure the user 5 is active state but user 1, user 2, user 3, and user 4 are in a waiting state whereas user 6 is in a ready state.

1. **Active State** - The user's program is under the control of the CPU. Only one program is available in this state.
2. **Ready State** - The user program is ready to execute but it is waiting for its turn to get the CPU. More than one user can be in a ready state at a time.
3. **Waiting State** - The user's program is waiting for some input/output operation. More than one user can be in a waiting state at a time.

Real Time Operating System

- A Real-Time Operating System (RTOS) is designed to process data and events with **strict timing constraints**, ensuring that critical tasks are completed within defined time limits.

- RTOS is a special kind of operating system designed to handle tasks that need to be **completed quickly and on time**. Unlike general-purpose operating systems (GPOS), which are good at multitasking and user interaction, RTOS focuses on doing things in real time.
- The main goal of an **RTOS is to perform critical tasks on time**. It ensures that certain processes are finished within strict deadlines, making it perfect for situations where timing is very important. It is also good at handling multiple tasks at once.
- The idea of real-time computing has been around for many years. The first RTOS was created by **Cambridge University in the 1960s.**
- It is **time-bound system** that can be defined as fixed time constraints. In this type of system, processing must be done inside the specified constraints. Otherwise, the system will fail.

Applications

- Automotive systems
- Industrial robots
- Medical equipment
- Aerospace (flight control systems)
- Consumer electronics (smart TVs, washing machines)
- IoT devices

Types of RTOS

Three types of RTOS systems are:

1. Hard Real Time

In Hard RTOS, the **deadline is handled very strictly** which means that given task must start executing on specified scheduled time, and must be completed within the assigned time duration.

Example: Medical critical care system, Aircraft systems, etc.

2. Firm Real time

These types of RTOS also need to follow the deadlines. However, **missing a deadline may not have big impact but could cause undesired affects**, like a huge reduction in quality of a product.

Example: Various types of Multimedia applications.

3. Soft Real Time

Soft Real time RTOS, accepts some delays by the Operating system. In this type of RTOS, there is a deadline assigned for a specific job, but a **delay for a small amount of time is acceptable**. So, deadlines are handled softly by this type of RTOS.

Example: Online Transaction system and Livestock price quotation System.

Distributed Operating System

A **distributed operating system** seamlessly connects multiple computers. These computers communicate and share resources like memory, storage, and processing power. Users see it as one large system, not separate machines.

How Does a Distributed Operating System Work?

- When a task is given, the system decides which computer will handle it. For example, if one computer is busy, the system can assign the task to another computer that has free resources. This ensures all computers are used efficiently without overloading any single one.

- The computers in the system talk to each other through a network. They exchange information to stay synchronized and complete tasks smoothly.

- Special protocols like TCP/IP handle this communication and make sure everything happens correctly. If there is an error, the system detects it and quickly takes steps to fix the problem.

Types of Distributed Operating Systems

There are three main types of distributed operating systems:

1. Client-Server Systems

A central server provides resources and services, while clients (users or devices) request and access these resources. The server manages data and processes, while the client interacts with the user.

Example: Online banking, where the central server processes transactions.

2. Peer-to-Peer Systems

In peer-to-peer (P2P) systems, all computers (peers) are equal and share responsibilities. Each peer can act both as a client and a server, sending and receiving data.

Example: File-sharing platforms like BitTorrent distribute files among users.

3. Clustered Systems

Clustered systems consist of a group of computers that work together to perform tasks as a unified system. These systems are often used for high-performance computing and are designed to handle intensive workloads.

Example: Supercomputers used for weather forecasting and scientific research.

Features of Distributed Operating Systems

1. Resource Sharing

- Computers share hardware and software resources.
- Shares files, printers, and memory across devices.
- Allows users to access resources from any connected computer.

2. Transparency

- It hides the complexity of multiple systems.
- Makes users feel like they are working on one machine.
- It hides details like the location of resources or data.

3. Fault Tolerance

- Continues working even if one machine fails.
- Ensures data is safe and accessible.

4. Scalability

- Easily adds more computers to the system.
- It handles large workloads efficiently.

5. Concurrency

- Many tasks are processed at the same time.
- This speeds up operations and improves productivity.

Mobile OS (Android OS)

- Android is an operating system based on a modified version of the Linux kernel and other open-source software, designed primarily for touchscreen-based mobile devices such as smartphones and tablet computers.
- Android has historically been developed by a consortium of developers known as the Open Handset Alliance, but its most widely used version is primarily developed by Google.
- First released in 2008, Android is the world's most widely used operating system.
- It is the most used operating system for smartphones, and also most used for tables, the latest version, released on June 10, 2025, is Android 16.

Key Features of Android

- **Open Source**: Android is an open-source platform, which means that anyone can access and modify the source code. This allows for a high degree of customization and flexibility.
- **Wide Range of Applications**: Android supports a vast array of applications, including those for entertainment, productivity, communication, and more.

- **Connectivity**: Android supports various connectivity options such as GSM, CDMA, Wi-Fi, Bluetooth, and more.
- **Multimedia Support**: Android supports a wide range of media formats for audio, video, and images.
- **Multitasking**: Android allows users to run multiple applications simultaneously and switch between them easily.

1.3 Command line based Operating System

- A **Command-Line Based Operating System** is an operating system where the **primary way to interact** with the computer is by typing **text commands** into a command-line interface (CLI), rather than using graphical icons or windows.
- A command-line interface (CLI) is a text-based user interface (UI) used to run programs, manage computer files and interact with the computer.
- Command-line interfaces are also called *command-line user interfaces*, *console user interfaces* and *character user interfaces*.
- CLIs accept as input commands that are entered by keyboard; the commands invoked at the command prompt are then run by the computer.

Key Features

1. **Text-Based Interface** – Users type commands instead of clicking buttons.
2. **Low Resource Usage** – No heavy graphics, so it runs fast even on old hardware.
3. **High Control & Flexibility** – Advanced users can perform complex tasks quickly.
4. **Scriptable** – You can automate tasks using scripts (batch files, shell scripts).
5. **Multiuser & Multitasking Support** – Many command-line systems can handle multiple users and processes simultaneously.

Examples: DOS,UNIX

GUI based Operating System

- A **GUI-Based Operating System** (Graphical User Interface Operating System) is an operating system where users interact with the computer through **visual elements** such as windows, icons, menus, and buttons, instead of typing text commands.
- These systems follow the **Model-View-Controller (MVC)** design pattern, separating the internal logic from the user interface.
- This allows users to perform tasks like file management, application launching, and system configuration through simple clicks and drags.
- The graphical user interface (GUI) is very easy to use and the user can modify and simplify the requirements.
- The required software, documents, or a few relevant programs are reflected in the icons on the user interface to control the overall processes properly.

Examples

- Microsoft Windows (Windows 10, 11, etc.)

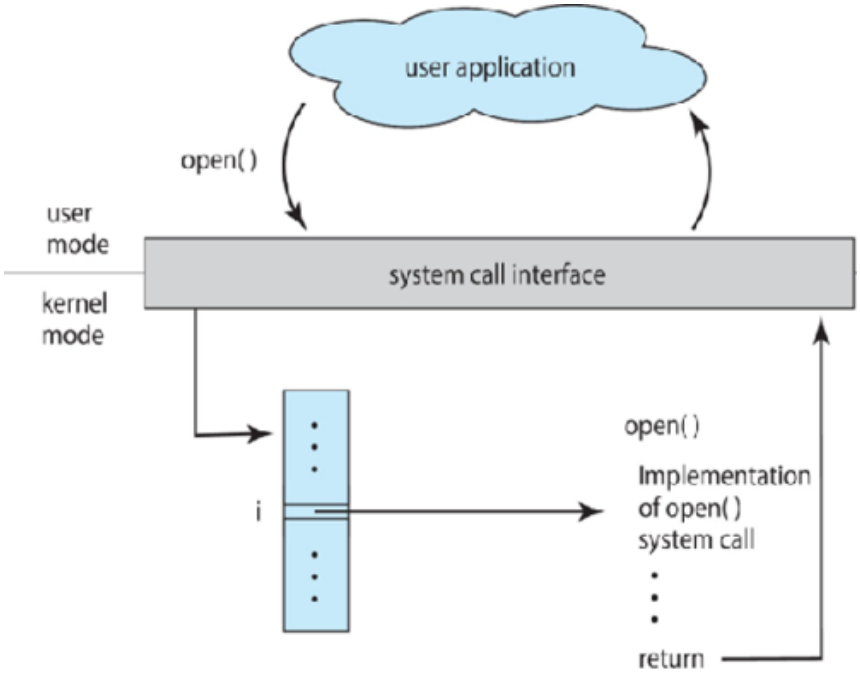
- macOS (Apple)
- Linux with desktop environments

1.4 Different Services of Operating System

1. **User interface:**
   Almost all operating systems have a User interface. This interface can take several forms. One is **Command Line Interface** which uses text commands and a method for entering them. Most commonly, a **Graphical user Interface** is used. Here, the interface is a window system with a pointing device to direct I/0, choose from menus, and make selections and a keyboard to enter text.
2. **Program execution:**
   The system must be able to load a program into memory and to run that program. The program must be able to end its execution, either normally or abnormally (indicating error).
3. **I/O operations**:
   A running program may require I/0, which may involve a file or an I/0 device. For efficiency and protection, users usually cannot control I/0 devices directly. Therefore, the operating system must provide a means to do I/0.
4. **File-system manipulation**:
   Creates, deletes, reads, writes, and manages files/directories. Controls access permissions to files (read/write/execute). Maps files onto secondary storage (like HDD, SSD).
5. **Communications**:
   There are many circumstances in which one process needs to exchange information with another process. Such communication occur between processes that are executing on the same computer or between processes that are executing on different computer systems tied together by a computer network. Communications may be implemented via shared memory or through message passing, in which packets of information are moved between processes by the operating system.
6. **Error detection**:
   The operating system needs to be constantly aware of possible errors. Errors may occur in the CPU and memory hardware (such as a memory error or a power failure), in I/0 devices, and in the user program. For each type of error, the operating system should take the appropriate action to ensure correct and consistent computing.
7. **Resource allocation**:
   When there are multiple users or multiple jobs running at the same time, resources must be allocated to each of them. Many different -types of resources are managed by the operating system. Some (such as CPU cycles, main memory, and file storage) may have special allocation code, whereas others (such as I/0 devices) may have much more general request and release code.
8. **Accounting:**
   Keeps track of how much resources (CPU, memory, disk, etc.) each user or job consume. Helps in billing (in time-sharing systems) and monitoring.
9. **Protection and security**.
   Prevents unauthorized access to resources and data. Ensures **process isolation** (one process doesn't interfere with another). Implements **authentication (passwords, biometrics, etc.)** and **encryption**.

System Calls

- When an application needs to **interact with hardware or the OS** (like reading a file, creating a process, or sending data over the network), it cannot directly access hardware. Instead, it makes a **system call**, which transfers control to the OS kernel.
- A **system call** is the fundamental way in which a user-level program requests a service from the operating system's kernel.
- **system call** provides an **interface between user programs and OS**.



Types of System Calls

1. Process control

- **end, abort:** A running program needs to able to halt its execution either normally (end) or abnormally (abort).
- **load, execute:** A process executing may want to load and execute another program.
- **create process, terminate process:** The new process is created by fork( ) system call and terminate a process by using exit( ) system call.
- **get process attributes, set process attributes :** When user/system creates a new process , it should be able to control its execution. This requires set of the attributes including job priority, total execution time, and so on. This is done with the help of get process attributes, set process attributes system calls.
- **wait for time , wait event, signal event :** During the execution of process, some event or interrupt occurs at that time wait for time , wait event, signal event system call used.
- **allocate and free memory:** When the process is created, memory is allocated and when the process terminate then memory is deallocated.

2. File management

- **create file, delete file:** Creating a new file with the help of create file system call. Once created file is no longer used, we can delete file by using delete file system call.
- **open, close file:**Once the file is crated, we need to open the file and use it. After using that file, we need to close the file.
- **read, write, reposition:** Once we open the file read the contents from the file, write the contents to file and reposition(change the position of file in the memory location).
- **get and set file attributes:** System requires the name of file and some attributes such as type of file, protection code, accounting information, date and time of file creation. This will get with the help of get and set file attributes.

3. Device management

- **request device, release device:** Process may need several devices while executing. Process will request devices through the request device system call and after utilizing the resource process will release the devices through release devices.
- **read, write, reposition:** Once the device has been requested we can read, write and reposition(sequential and direct access) the device.
- **get device attributes, set device attributes:** System requires the name of devices and other information of devices (device queue, total time required for particular process). This will get with the help of get device attributes, set device attributes.
- **logically attach or detach devices:** Once the requested device is available, it is logically attach with the process. Once the process terminates the device is removed (logically detach).

4. Information maintenance

- **get time or date, set time or date:** Many system calls exist simply for the purpose of transferring information between the user program and operating system. Some system call return the current time & date of the system (get time and set time).
- **get system data, set system data:** These system call return or set the system related data such as no. of current users, version of Operating system, amount of free memory.
- **get and set process, file, or device attributes :** The OS will keeps the information about processes and files like creation time of process or files, memory required for process and files, etc. This will done through get and set process, file, or device attributes.

5. Communications

- **create, delete communication connection :** In message passing, to exchange the messages we need to establish communication link between sender and receiver by

using create communication connection system call. After the communication, OS will close the connection using delete communication connection system call.

- **send, and receive messages:** After creation of connection sender will send the message by using send( ) system call and receiver will receive the message by using receive ( ) system call.
- **transfer status information:** This system call will provide the status information of the system whether it is idle or busy.
- **attach and detach remote devices:** For accessing data from remote devices, the devices are need to be connected with the help of attach remote devices system call and disconnected through detach remote devices system call.

6. Protection

- **get permission , set permission:** These system calls provides the mechanism for protection and which manipulate the permission settings.
- **allow user , deny user:** These system call specify whether user can allowed or denied for accessing various resources.

Operating System components:

An operating system provides the environment within which programs are executed. To construct such an environment, the system is partitioned into small modules with a well-defined interface.

1. Process Management

- A program does nothing unless its instructions are executed by a CPU. A program in execution is called process.
- A time-shared user program such as a compiler is a process. A word-processing program being run by an individual user on a PC is a process. A system task, such as sending output to a printer, can also be a process
- A process needs certain resources including CPU time, memory, files, and I/O devices to accomplish its task. These resources are either given to the process when it is created or allocated to it while it is running.

The operating system is responsible for the following activities in connection with processes managed.

- The creation and deletion of both user and system processes
- The suspension, resumption of processes.
- The provision of mechanisms for process synchronization and communication
- The provision of mechanisms for deadlock handling.

1. Main Memory Management

- Main memory is a **large array of words or bytes**, ranging in size from hundreds of thousands to billions.
- Main memory is a repository of quickly accessible data shared by the CPU and I/0 devices. The processor reads instructions from main memory during **the instruction-fetch cycle** and both reads and writes data from main memory during the **data-fetch cycle** .
- The main memory is generally the only large storage device that the CPU is able to address and access directly. For example, for the CPU to process data from disk, those data must first be

transferred to main memory by CPU-generated I/0 calls. In the same way, instructions must be in memory for the CPU to execute them.

- For a program to be executed, it must be mapped to absolute addresses and loaded into memory.

The operating system is responsible for the following activities in connection with memory management.

- Keep track of which parts of memory are currently being used and by whom.
- Decide which processes are to be loaded into memory when memory space becomes available.
- Allocate and deallocate memory space as needed.

2. Secondary Storage Management

- The main purpose of a computer system is to execute programs. These programs, together with the data they access, must be in main memory during execution.
- Since the main memory is too small to permanently accommodate all data and program, the computer system must provide secondary storage to backup main memory.
- Most modem computer systems use disks as the primary on-line storage of information, of both programs and data.
- Most programs, like compilers, assemblers, sort routines, editors, formatters, and so on, are stored on the disk until loaded into memory, and then use the disk as both the source and destination of their processing. Hence the proper management of disk storage is of central importance to a computer system.
- There are few alternatives. Magnetic tape systems are generally too slow. In addition, they are limited to sequential access. Thus tapes are more suited for storing infrequently used files, where speed is not a primary concern.

The operating system is responsible for the following activities in connection with disk management

- Free space management
- Storage allocation
- Disk scheduling.

3. I/O System

One of the purposes of an operating system is to hide the peculiarities of specific hardware devices from the user. For example, in Unix, the peculiarities of I/O devices are hidden from the bulk of the operating system itself by the I/O system. The I/O system consists of:

- A buffer caching system
- A general device driver code
- Drivers for specific hardware devices.

Only the device driver knows the peculiarities of a specific device.

File Management

File management is one of the most visible services of an operating system. Computers can store information in several different physical forms; magnetic tape, disk, and drum are the

most common forms. Each of these devices has its own characteristics and physical organization.

For convenient use of the computer system, the operating system provides a uniform logical view of information storage. The operating system abstracts from the physical properties of its storage devices to define a logical storage unit, the file. Files are mapped, by the operating system, onto physical devices.

A file is a collection of related information defined by its creator. Commonly, files represent programs (both source and object forms) and data. Data files may be numeric, alphabetic or alphanumeric. Files may be free-form, such as text files, or may be rigidly formatted. In general a files is a sequence of bits, bytes, lines or records whose meaning is defined by its creator and user. It is a very general concept.

The operating system implements the abstract concept of the file by managing mass storage device, such as types and disks. Also files are normally organized into directories to ease their use. Finally, when multiple users have access to files, it may be desirable to control by whom and in what ways files may be accessed.

The operating system is responsible for the following activities in connection with file management:

- The creation and deletion of files
- The creation and deletion of directory
- The support of primitives for manipulating files and directories
- The mapping of files onto disk storage.
- Backup of files on stable (nonvolatile) storage.